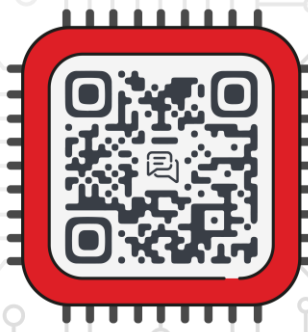




4

26

3140 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Files in Java/Java IO

د. محمد الأحمد



RB Informatics; 13/11/2024

برمجة ٣

السلام عليكم

سنتعلم في هذه المحاضرة :

Streams (1

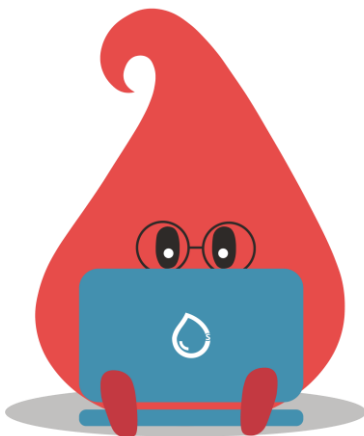
Java io (2

files (3

- سابقاً كنا نتبع طريقة إدخال المعلومات إما من لوحة المفاتيح أو الميكروفون أو الماوس، و يتم تخزينها مؤقتاً إما في متحولات variables أو في مصفوفات Arrays أو غيرها، ولكن كانت تموت بمجرد انتهاء البرنامج (الخروج من ال scope المعرفة بداخله) ولكن في بعض الأحيان نحتاج إلى أن تكون ال data دائمة (permanent)، فنتبع بعض طرق التخزين إما في files/ data Base.
- مثال : لدينا حساب لعميل في بنك وفي حال تم تخزين البيانات في متحولات فإنها ستموت المعلومات بانتهاء البرنامج ولن نستطيع التعديل على هذا الحساب أو معرفة البيانات مجدداً، لذلك من الممكن تخزين البيانات في file.

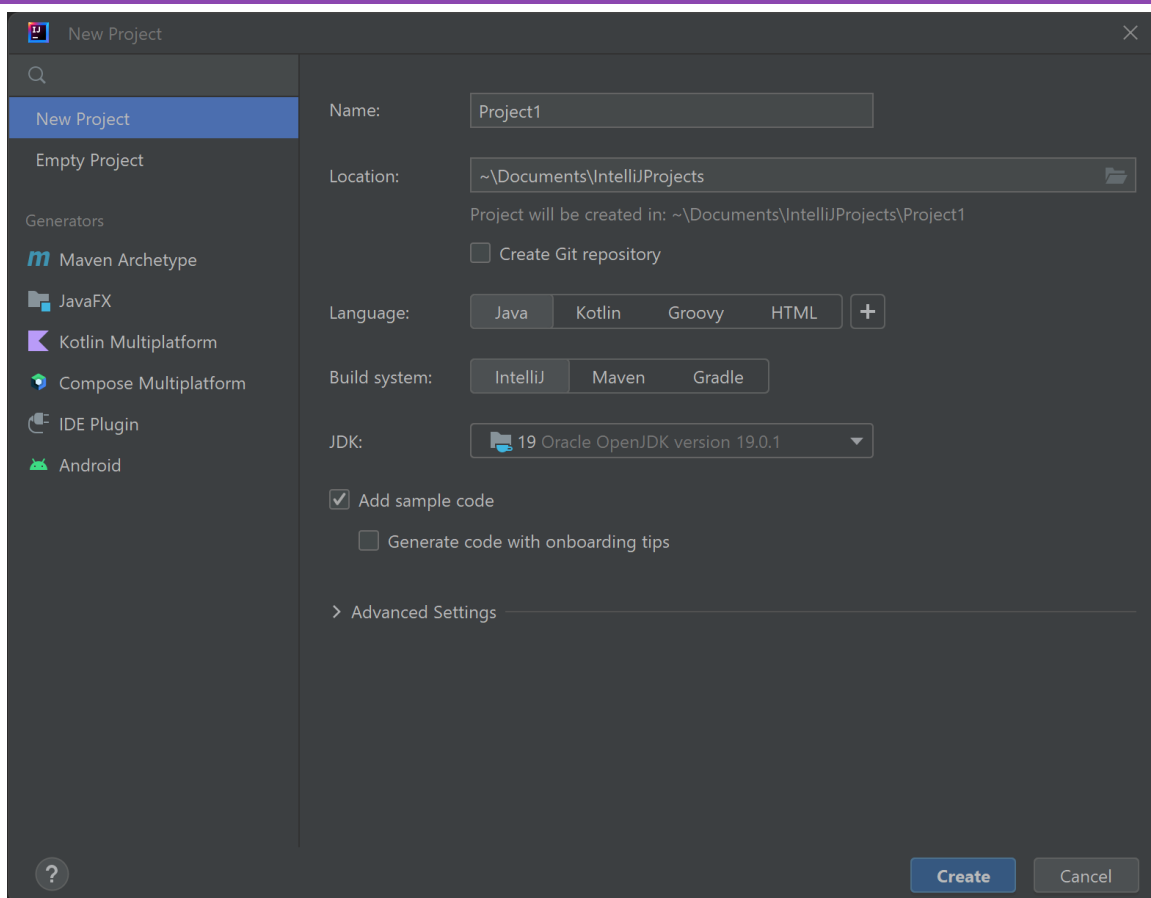
أهمية التعامل مع ال files في جافا :

- غالباً ما نحتاج تبادل البيانات بين المصادر الخارجية و البرامج فيمكننا تبادل البيانات بين البرنامج والملف أو البرنامج و الشبكة أو قواعد البيانات...

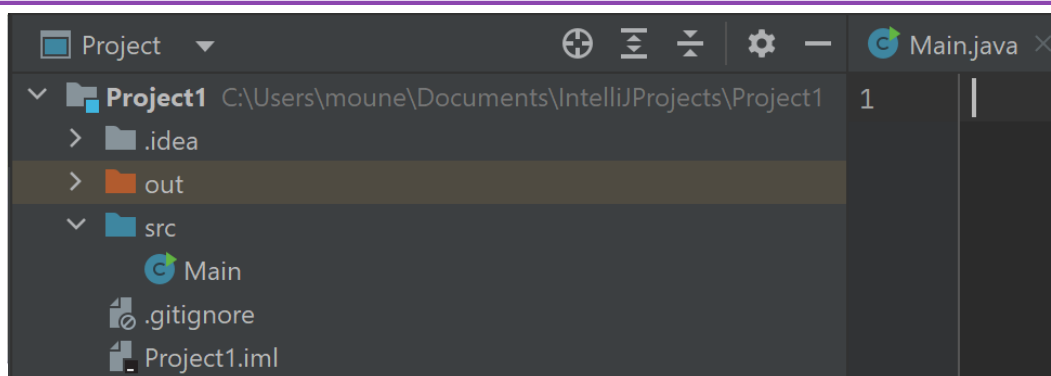


إضافة أريسيزية: كيفية التعامل مع الـ file في برنامج الـ intelij

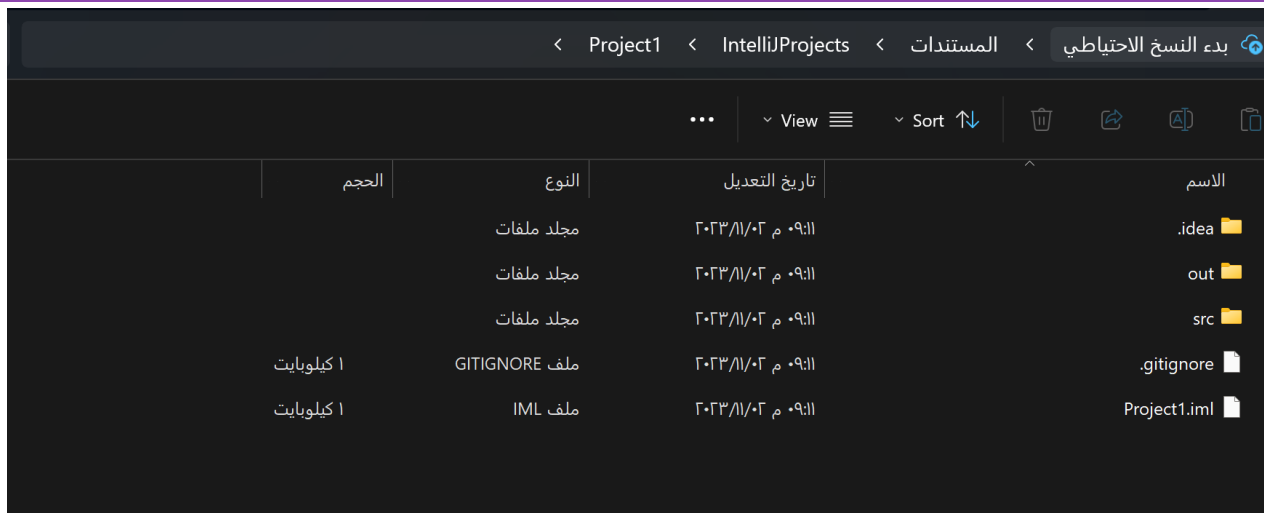
(1) عند فتح البرنامج يظهر لنا نافذة لإنشاء new project



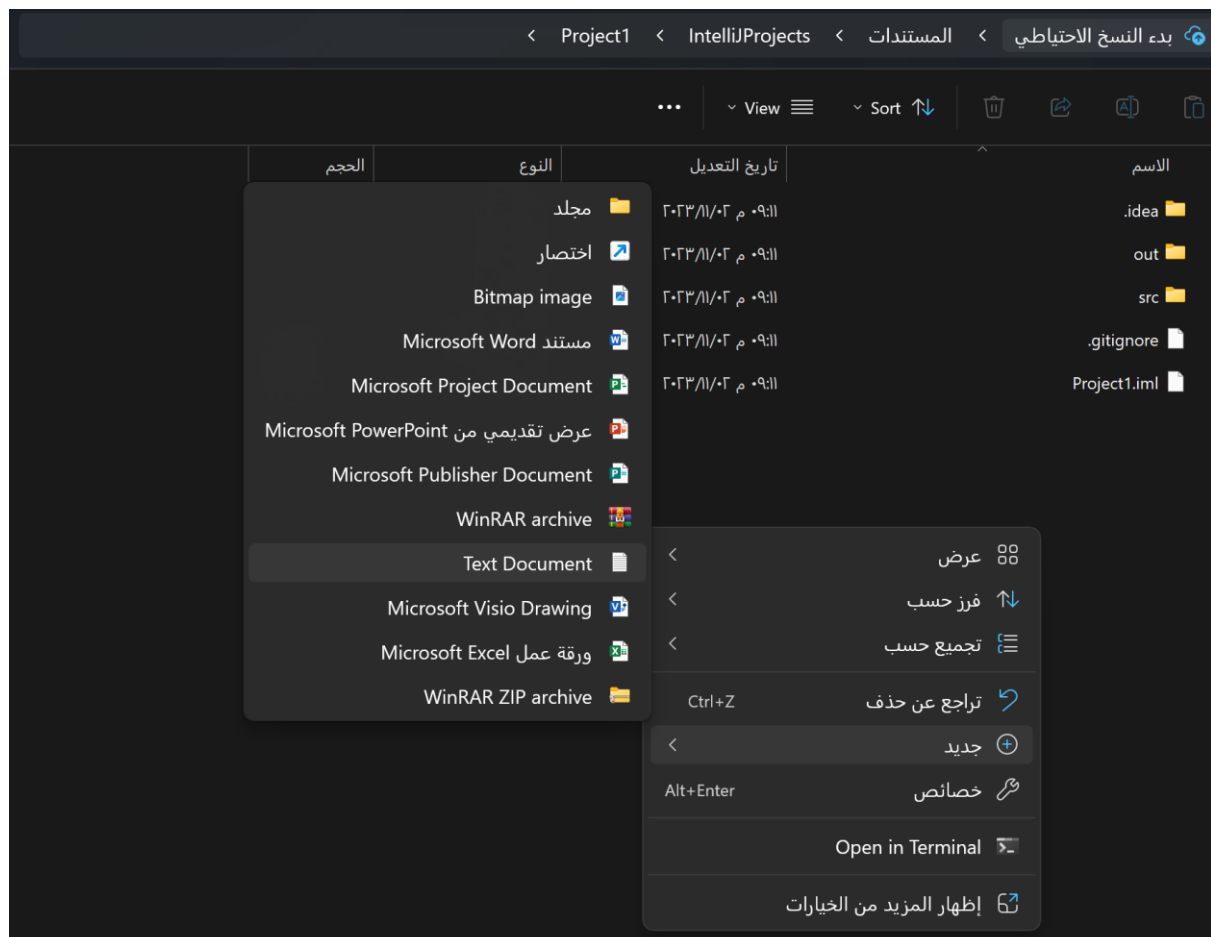
(2) عند إنشاء الـ project الجديد سيظهر لنا قائمة على اليسار بالكلاسات الموجودة و الملفات



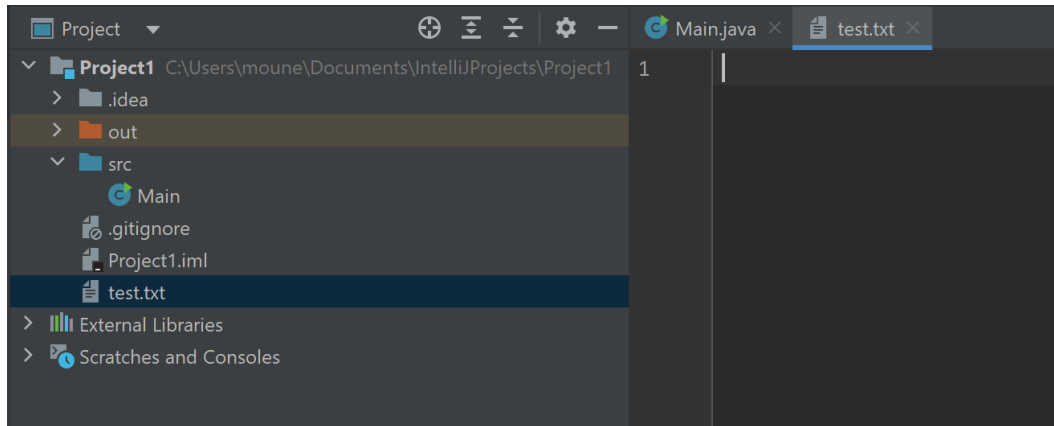
(3) تتبع المسار الموجود حيث يوجد ال classes و ال files



(4) قم بإنشاء ملف نصي و قم بتسميته



(5) عند العودة إلى نافذة البرنامج سيظهر لنا الملف النصي الذي قمنا بإضافته على اليسار



java.io:

- لغة الجافا توفر لنا الحزمة java.io والتي تحتوي على جميع الكلاسات التي تتعامل مع إدخال وإخراج البيانات ضمن هرمية وراثية كبيرة (large inheritance hierarchy)، وتحتوي كلاسات مخصصة للتعامل مع الملفات سواء لإدخال البيانات أو لاستخراجها منها، و io تعني input/output.
- import.java.io.class; استيراد الحزمة.
- التعامل مع البيانات كبايتات فردية محدود وغير كافٍ، لهذا تتضمن هذه الحزمة كلاسات تقدم وظائف إضافية كالbuffering وقراءة الأرقام والstrings بدلاً من التعامل معها كبايتات فقط.
- الإدخال والإخراج في جافا عمليات مستقلة عن النظام الأساسي (Java I/O designed to be platform-independent).

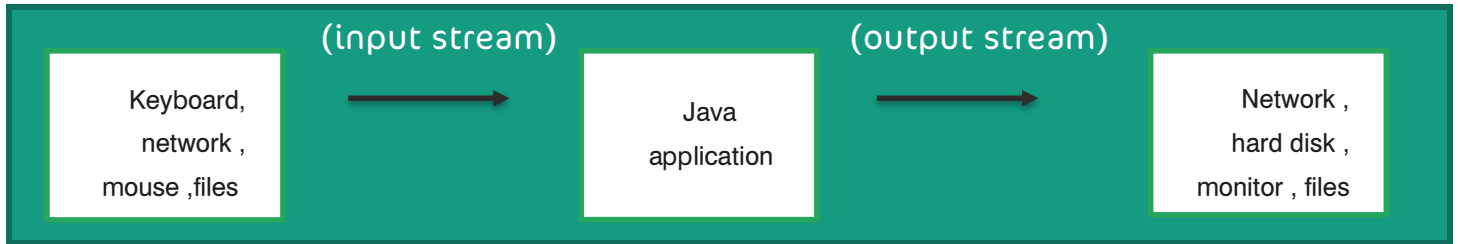
مفهوم ال stream في الجافا :

- Ordered sequence of data (سلسلة مرتبة من البيانات/دفق من البيانات) or information (bytes) coming from a source, or going to a 'sink/معيّنة'.
- In Java: Stream is an object that either delivers data to its destination (screen, file, etc.) or that takes data from a source (keyboard, file, etc.).
- Simplest stream reads/writes only a single byte, or an array of bytes at a time.
- Java is a stream oriented language (تعتمد جافا بشكل أساسي على مفهوم الستريم).
- The stream concept is very generic and can be applied to many different types of I/O (يمكن استخدام مفهوم الستريم للتعامل مع البيانات التي تُقرأ من مصادر أو أجهزة مختلفة أو تكتب فيها) تقوم بتجريد مفهوم الإدخال والإخراج
- Types of I/O sources: files, network, memory, processes, etc.

الهدف من ال stream :

نقل المعلومات بين البرنامج و ال device (I/O object)، وهي عبارة عن object (acts as a buffer between the data source and destination).

Input stream provides input to a program (من الجهاز للبرنامج), and Output stream accepts output from a program (من البرنامج للجهاز).



ملاحظة : هناك predefined streams مرت معنا وهي:

- System.in : an input stream (يمثل الدخل من لوحة المفاتيح)
- System.out : an output stream (يمثل الخرج على الشاشة)
- System.error : an output stream (عرض رسائل الخطأ)

تذكرة:

- إن جميع البيانات في نهاية المطاف تُمثل بأصفار ووحدات، حيث يأخذ كل digit إحدى هاتين القيمتين.
- ال Bit هو أصغر وحدة بيانات، ويمثل قيمة digit ثنائي واحد (أما 0 أو 1).
- ال Byte: هو مجموعة مكونة من 8 bits.

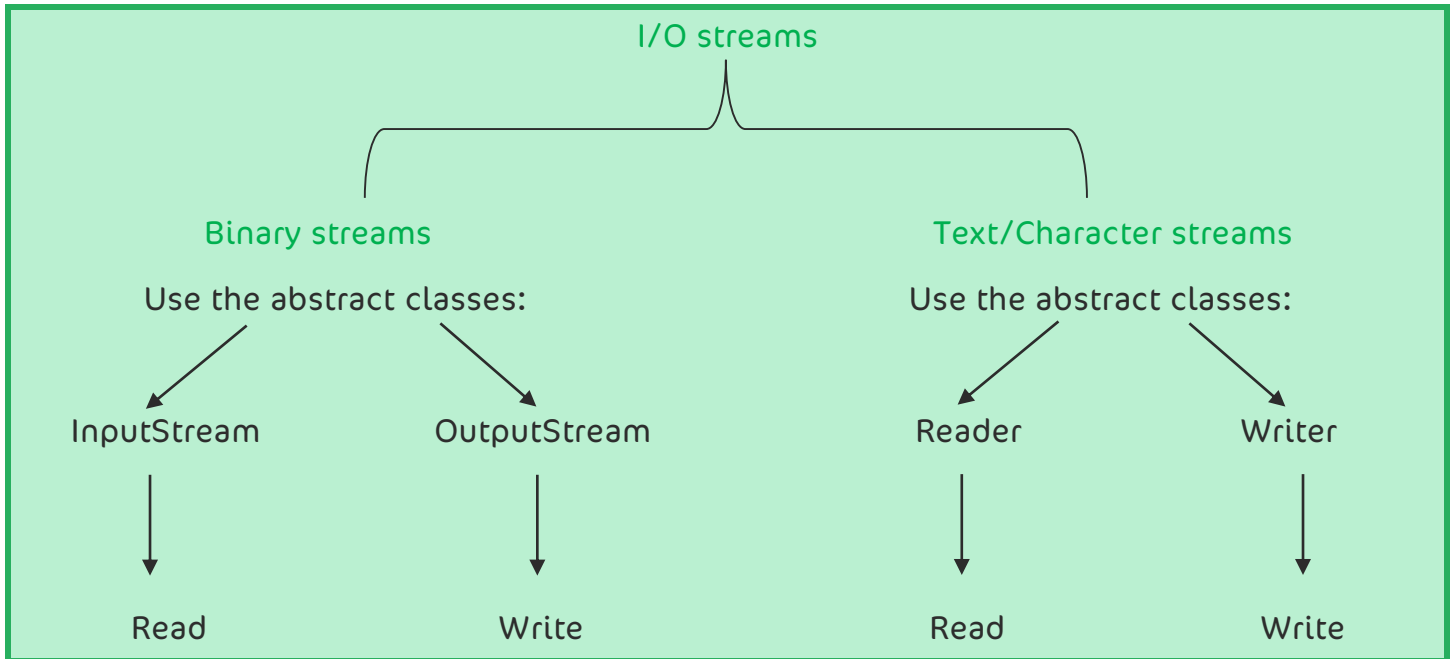
Types of files in java :

Binary files	Text files (Character files)
تمثل ال-bits فيها أنواعاً من المعلومات المشفرة (encoded information) مثل التعليمات القابلة للتنفيذ (executable instructions) والبيانات الرقمية (numeric data).	تمثل ال-bits فيها محارف قابلة للقراءة والطباعة والتحرير بسهولة.
سهولة القراءة بالنسبة للحاسوب وليس الإنسان (تمثل بـ 0 & 1).	قابلة للقراءة من قبل الإنسان (readable).
Not printable (في حال تمت طباعتها سيكون المحتوى غير قابل للفهم-unintelligible)	Printable (في حال تمت طباعتها سيكون المحتوى قابلاً للفهم)
غير نصية تخزن بشكل سلسلة من ال-Bytes، وتتم معالجة البيانات كـ byte واحد لكل قيمة.	نصية تعتمد على ترميز Unicode، أي أن كل محرف يتم تمثيله بـ 2 Bytes، و ستتطرق للحديث عنه بالتفصيل.

من أمثلتها: الصور/ النصوص/ الفيديوهات/ الصوت (معظم الملفات)	من أمثلتها: Java source files / أي ملف يتم إنشاؤه باستخدام text editor .
تتعامل معها الكلاسات التي تصنف كـ Binary streams	تتعامل معها الكلاسات التي تصنف كـ character streams

الفرق بين الـ :

Unicode	Ascii
معيّار أوسع يمكنه تمثيل جميع الأحرف بأي لغة.	معيّار لترميز الأحرف و الرموز اللاتينية.
يعتمد 2 bytes (16 bites)	يعتمد 1 byte
يكفي لجميع اللغات المعروفة تقريباً وحتى بعض اللغات الخيالية! (...Klingon, Elvish)	لا يكفي للأبجديات المعقدة كاللغة الصينية
تتعامل مع java مع النص باستخدامه (المحرف فيها 2bytes) وتحتفظ الـ JVM بالنصوص بواسطته.	تتعامل مع ++c مع النص باستخدامه (المحرف فيها 1byte)
من الممكن أن يستخدم 8bits أو 16 أو 32 لتقديم أي حرف	نظام ترميز 7 bits أي 128 محرف (الأحرف اللاتينية مع المحارف الطباعية وغير الطباعية) من 0 ← 127



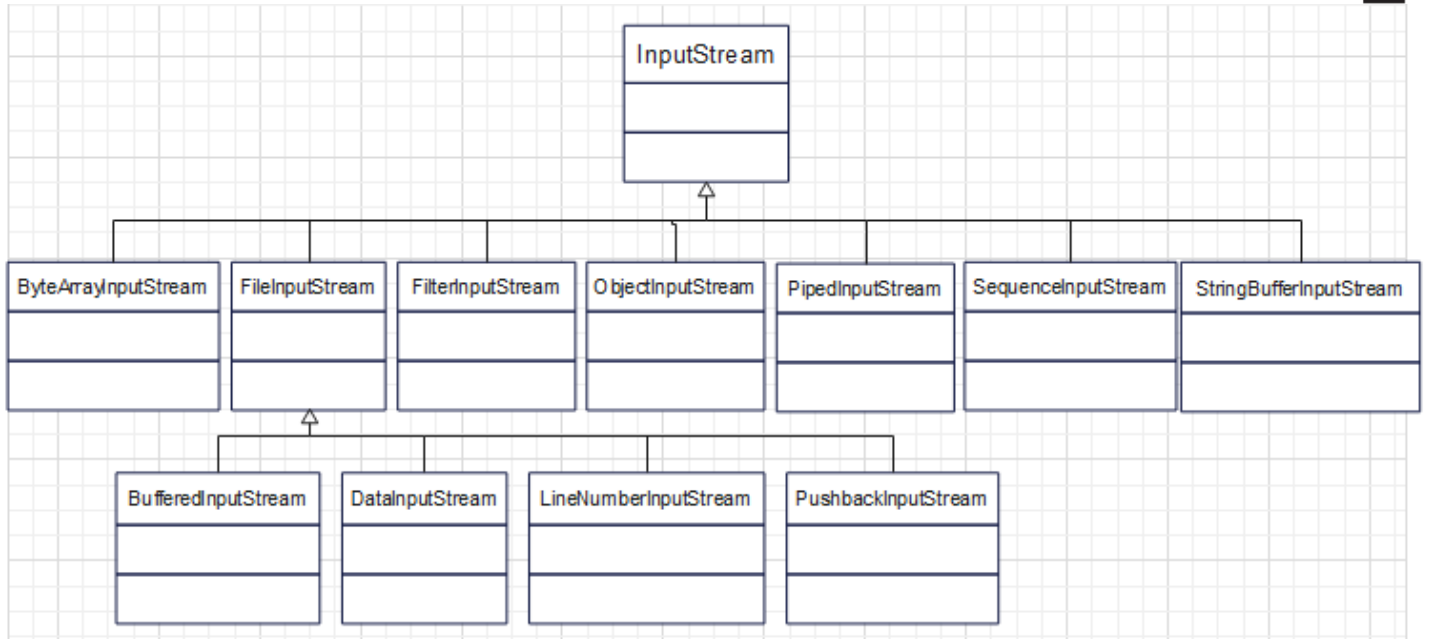
- الـ binary أكثر فاعلية من الـ text لأنها تستخدم الـ machine language (0/1) مباشرة بدلاً من الترجمة.

1. InputStream :

هو الكلاس الأساسي لقراءة البيانات من الملف ويحتوي على العديد من الـ methods.

- بعض الـ methods المجردة الموجودة ضمنه :

available() : int	للاستعلام عن عدد البايتات المتاحة للقراءة من دون انتظار (تعيد قيمة تمثل العدد التقريبي)
close : void	تستخدم لإغلاق المصادر المرتبطة بالـ input stream ، وهي ضرورية (عدم إغلاقها يسبب ظهور exception).
mark (limit : int) : void	تستخدم لتحديد موضع القراءة الحالي، مما يتيح لك الرجوع إليه لاحقاً باستخدام reset()، وقيمة limit تحدد عدد البايتات التي يمكن تخزينها بعد العلامة قبل أن تُفقد
reset() : void	تستخدم لإعادة تعيين موضع القراءة إلى الموضع الذي تحدده mark()، وعدم استدعائها بعد mark() يؤدي لـ IOException
markSupported() : boolean	تستخدم للتحقق مما إذا كان InputStream يدعم عملية mark() و reset()
read() : int	تستخدم لقراءة البيانات من الـ InputStream وتقوم بإرجاع بايت من المدخل كرمز بيني (byte) من النص
read (b : byte[]) : int	يتم قراءة البيانات إلى مصفوفة البايتات ويعيد العدد الإجمالي للبايتات التي تمت قراءتها
read (b: byte[], offset: void, length : void) : int	تستخدم لقراءة جزء من مصفوفة البايتات (b) بدءاً من الـ index (offset) بطول (length)
skip (l : long) : long	تستخدم لتخطي وتجاوز عدد محدد من الأحرف وتعيد هذا العدد



ملاحظات :

- بما أن الكلاسات (InputStream, OutputStream, Reader, Writer) جميعها كلاسات مجردة (abstract) فيتم التعامل مع الأبناء وإنشاء كائنات منها.
- أهم كلاس للتعامل مع الـ input stream هو FileInputStream، ويوفر العديد من الـ methods للتعامل مع الملفات بسهولة.
- نلاحظ أن كل كلاس من الأبناء يمثل device (المقصود مصدر لتخزين البيانات كالشبكات والمصفوفات والملفات و...)، فنستخدم أحدها بحسب المصدر المراد القراءة منه أو الكتابة إليه.
- عملية الـ Serialization: تعني استخدام الـ object للقراءة أو الكتابة (ObjectInputStream / ObjectOutputStream).

إضافة أريسيزية:

- **ObjectInputStream:**
يستخدم لقراءة كائنات تم حفظها إلى ملف أو إلى الشبكة، ويكون تخزينها بشكل binary (غير مفهوم من قبل المستخدم).
- **FilteredInputStream:**
يستخدم لإنشاء streams إدخال "مفلترة" يمكن تعديل سلوكها أو تطبيق وظائف إضافية عليها، وذلك عن طريق إنشاء كلاس يرث من FilteredInputStream ثم عمل Override لميثود read() والتعديل على البيانات المقروءة (غالباً باستخدام عبارات شرطية)، فينتج كلاس يمكن استخدامه كأبي stream.

خطوات استخدام الـ streams :

1. فتح الـ stream.
2. طالما هناك بيانات للقراءة/للكتابة: معالجة التعليمات .
3. إغلاق المصدر.

■ ملاحظة: هذه الخطوات هي standard (ثابتة) في معظم versions جافا، ولكن كل version تضيف إليها أو تطور الـ IO.

مثال :

```

1. // استيراد الحزم المناسبة
2. import java.io.InputStream;
3. import java.io.FileInputStream;
4. import java.io.IOException;
5.
6. public class Main {
7.     public static void main(String[] args) {
8.         // exception handling قمنا بـ
9.         InputStream in = null;
10.        try {
11.            // عندما تستخدم هذه الكلاسات ستستخدم مبدأ
Upcasting يسمى
12.            // المقصود من هذا أنك مثلاً تنشئ كائناً من
أحد الأبناء (هنا FileInputStream) ثم تسنده إلى
InputStream من reference
13.            // مع تحديد مسار الملف واسمه
14.            in = new FileInputStream
("d:\\users\\myfile\\test.txt");
15.
16.            // القراءة من الملف
17.            int b = in.read();
18.
19.            // إنشاء حلقة بحيث تنكسر عندما b == -1
أي عندما ينتهي الملف
20.            while (b != -1) {
21.                // إجراءات معالجة البيانات المقروءة
22.                b = in.read();
23.            }
24.        } catch (IOException e) {
25.            System.out.println("Error
reading from file: " + e.getMessage());
26.        }
27.        // إغلاق المصدر للحفاظ على أمنه
28.        finally {
29.            try {
30.                in.close();
31.            } catch (IOException e) {
32.                System.out.println("Error
closing the stream: " + e.getMessage());
33.            }
34.        }
35.    }
36. }

```

Failure will never overtake me if my
determination to succeed is strong enough.

- Og Mandino.



1. cin و cout في ++c هي streams (فعلياً objects).
2. تقريباً جميع methods والconstructors في كلاسات I/O تقوم بإلقاء IOException أو أحد أبنائه، وهو checked exception ، لذا: we always wrap I/O code in try...catch blocks.
3. المسار في الكود السابق يعني أن الملف الذي يُقرأ منه اسمه test ، موجود في مجلد myfile ضمن المجلد users ضمن d.
- في المثال السابق وضعنا double backslash (\\) slash واحدة ولا يتم أخذها مع ما بعدها ك escape sequence لأنها ضمن double quotation marks ("")، أي عملياً سيقرأ المسار هكذا: d:\users\myfile\test.txt.
4. هناك نوعان من المسارات (paths) لأي ملف:
 - absolute path: يشير لموقع الملف من بداية الـ directory حتى نهاية العنوان (الملف المستهدف)، فيتضمن جميع المجلدات التي يجب المرور بها للوصول إليه، وفي الكود السابق مثال عنه.
 - relative path: يشير لموقع الملف بالنسبة للـ (Current working directory)، أي يعتمد على موقع الملف الحالي دون الحاجة لكتابة المسار الكامل.
5. إن الـ read method في كلاس FileInputStream تستخدم لقراءة عدد من البايتات وتخزينه في متحول من نوع int، لذا قمنا بتعريف المتحول b في السطر 17 كمتحول من نوع int، ومن الممكن إن استخدمنا كلاسات أخرى أن نستخدم نوعاً آخر كالـ reference type أو primitive type مختلف (char, double, ...etc).
6. بما أن في الـ file مجموعة محددة من البيانات فهذا يعني أن لديه نهايةً دوماً (بعكس الإدخال من لوحة المفاتيح)، هذه النهاية تدعى EOF (End Of File)، نعبّر عنها بعدة طرق، وفي المثال السابق لدينا الميثود read() -1 عندما تنتهي من قراءة الملف، لذا جعلنا شرط كسر الحلقة أن تكون b=-1، وسنرى في أمثلة لاحقة أننا نستخدم شروطاً مختلفة في حالات أخرى.

مفهوم ال Buffer :

هو منطقة في الذاكرة (cache) تستخدم لتخزين البيانات مؤقتاً أثناء نقلها من مكان إلى آخر، ويستخدم لقراءة وكتابة البيانات (هناك نوع للإدخال ونوع للإخراج) خاصة عند التعامل مع الملفات وعمليات الإدخال والإخراج أو الاتصال عبر الشبكة، حيث تتيح methods أكثر وتساعد في تحسين الأداء وجعله أكثر سرعة وفاعلية (extra functionality)، فيمكن قراءة أو كتابة البيانات بشكل مجموعة (وليس بشكل فردي)، أي استخدام أكثر من stream معاً وبشكل متداخل (nested streams)، ويتم التخلص منها (إرسالها للوجهة المحددة/الملف/الوجهة المستخدم...إلخ) عند الانتهاء (باستخدام الميثود flush).

مثال لدعم الفكرة :

- عند وجود ملف بحجم 10 MB هل يجب استدعاء الميثود أكثر من 10 ملايين مرة ؟
- باستخدام مفهوم ال Buffer يمكننا قراءة 1000 Bytes أو أكثر معاً بدل من استدعائها لقراءة 1 Byte و ذلك يحسن الأداء من حيث السرعة ومساحة التخزين .
- لتطبيق هذا المفهوم هنا نحتاج لإنشاء كائن من كلاس BufferedInputStream (أحد أبناء كلاس FileInputStream) وإعطائه reference من FileInputStream.

```

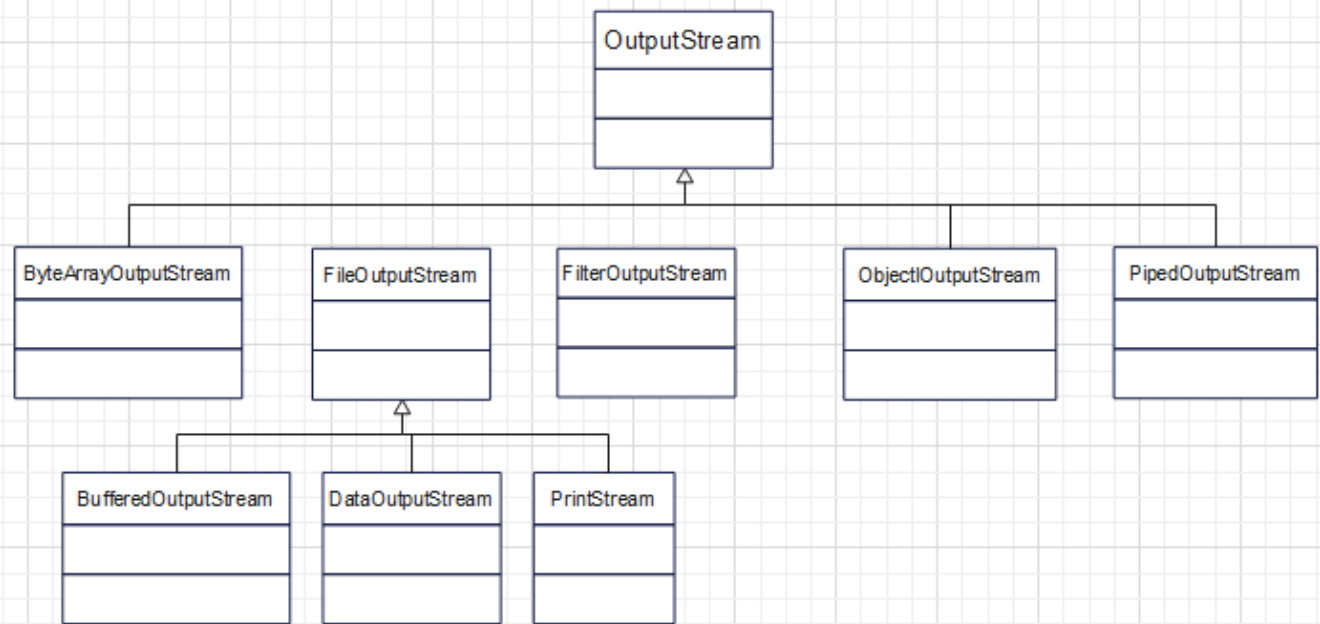
1 //استيراد الحزم المناسبة
2 import java.io.FileNotFoundException;
3 import java.io.InputStream;
4 import java.io.FileInputStream;
5 import java.io.IOException;
6 import java.io.FileOutputStream;
7 import java.io.BufferedInputStream;
8
9
10 public class Main {
11     public static void main(String[] args) {
12         InputStream in = null;
13         try {
14             //القراءة من الملف
15             InputStream inner = new FileInputStream
16                 ("c:\\temp\\myfile.txt");
17             in = new BufferedInputStream
18                 (inner);
19             //process file
20         } catch (IOException e) {
21             e.printStackTrace();
22         } finally {
23             try {
24                 in.close();
25             } catch (Exception e) {
26             }
27         }
28     }
29 }

```

2. OutputStream :

هو الكلاس الأساسي لكتابة البيانات في ملف، و أيضاً يحتوي على ال methods :

close() : void	إغلاق المصادر
write(b : byte) : void	كتابة بايت واحد من البيانات
write (b : byte[]) : void	كتابة مصفوفة من البايتات
write (b: byte[], offset: int, len : void): void	كتابة جزء من مصفوفة البايتات (b) يبدأ من ال index (offset) بطول (len)
flush() : void	تفريغ أي بيانات معلقة في الذاكرة المؤقتة وضمان أن جميع البيانات قد تمت كتابتها



إضافة آريسيزية:

- **ObjectOutputStream:**
يستخدم لحفظ كائنات إلى ملف أو إلى الشبكة، ويكون تخزينها بشكل binary .
- **FilteredOutputStream:**
يستخدم لإنشاء streams إخراج "مفلتر" يمكن تعديل سلوكها أو تطبيق وظائف إضافية عليها، وذلك بنفس الطريقة التي تم عرضها في FilteredInputStream ولكن باستخدام الميثود write() .

■ مثال آريسيزي :

```

1. import java.io.FileOutputStream;
2. import java.io.IOException;
3. import java.io.OutputStream;
4.
5. public class Main {
6.     public static void main(String[] args) {
7.         OutputStream out = null;
8.         try {
9.             // استخدام مفهوم الـ upcasting
10.            // تحديد مسار الملف واسمه
11.            out = new FileOutputStream
12.            ("d:\\users\\myfile\\test.txt");
13.
14.            // كتابة بيانات إلى الملف
15.            String data = "Hello, World!\nThis is a
16.            test for FileOutputStream.";
17.
18.            // تحويل النص إلى مصفوفة من البايتات
19.            byte[] dataBytes = data.getBytes();
20.
21.            // كتابة المصفوفة إلى الملف
22.            out.write(dataBytes);
23.
24.            // إغلاق المصدر للحفاظ على الموارد
25.            catch (IOException e) {
26.                System.out.println(e);
27.            }
28.
29.            finally {
30.                try {
31.                    out.close();
32.                } catch (IOException e) {
33.
34.                }
35.            }
36.        }
37.    }
38. }
  
```

أيضاً تنقسم إلى نوعين من الكلاسات وهما Reader و Writer ، وأكثر الكلاسات استخداماً هما FileReader و FileWriter ، حيث يقوم الكلاس FileReader بقراءة حرف (2 Bytes) أو مجموعة من الأحرف في كل مرة، ويقوم الكلاس FileWriter بكتابة حرف (2 Bytes) أو مجموعة من الأحرف، وهو يعتمد على ترميز Unicode كما تحدثنا سابقاً.

■ ملاحظة: هذه الكلاسات تحول البايتات إلى محارف، وهذا يحل مشكلة أن `byte != character` في جميع اللغات، فهي تتعامل مع المحارف بدلاً من البايتات.

■ بعض الـ methods ضمن الكلاس Reader:

close : void	تستخدم لإغلاق المصادر
mark (limit : int) : void	تستخدم لتحديد موقع معين يمكن استرجاعه باستخدام <code>reset(). limit</code>
reset() : void	تستخدم لإعادة تعيين موضع القراءة إلى الموضع الذي تحدده <code>mark()</code> ، وعدم استدعائها بعد <code>mark()</code> يؤدي لـ <code>IOException</code>
markSupported() : boolean	تستخدم للتحقق مما إذا كان Reader يدعم عملية <code>reset()</code> و <code>mark()</code>
read() : int	تستخدم لقراءة البيانات وتقوم بإرجاع بايت من المدخل كرمز بيني (byte) من النص
read (c : char[]) : int	يتم قراءة البيانات إلى مصفوفة المحارف ويعيد العدد الإجمالي للمحارف التي تمت قراءتها
read (c: char[], off : int, len : int): void	تستخدم لقراءة جزء من مصفوفة المحارف (c) بدءاً من <code>index (off)</code> بطول (len)
skip (arg 1 : long) : long	تستخدم لتخطي وتجاوز عدد محدد من الأحرف وتعيد هذا العدد

■ مثال باستخدام الكلاس Reader :

```

1 import java.io.IOException;
2 import java.io.Reader;
3 import java.io.BufferedReader;
4 import java.io.FileReader;
5
6
7 public class Main {
8     public static void main(String[] args) {
9         //كود الـ Reader
10        Reader in = null;
11        try
12        {
13            Reader inner = new FileReader
14            ("c:\\temp\\myfile.txt");

```

```

14        in = new BufferedReader(inner);
15        //process file
16        } catch (IOException e)
17        {
18            e.printStackTrace();
19        }
20        finally
21        {
22            try { in.close(); } catch (Exception e) {}
23        }
24    }
25 }

```

■ بعض الـ methods ضمن الكلاس Writer:

close() : void	إغلاق المصادر
write(c : char) : void	كتابة حرف واحد من البيانات
write (c : char[]) : void	كتابة مصفوفة من المحارف
write (s : String) : void	كتابة نص
write (s: String, off: int, len : int): void	كتابة جزء من الـ String (s) يبدأ من الـ index (off) بطول (len)
write (c: char[], off: int, len : int): void	كتابة جزء من مصفوفة المحارف (c) يبدأ من الـ index (off) بطول (len)
flush() : void	تفريغ أي بيانات معلقة في الذاكرة المؤقتة وضمان أن جميع البيانات قد تمت كتابتها

■ مثال باستخدام الكلاس Writer

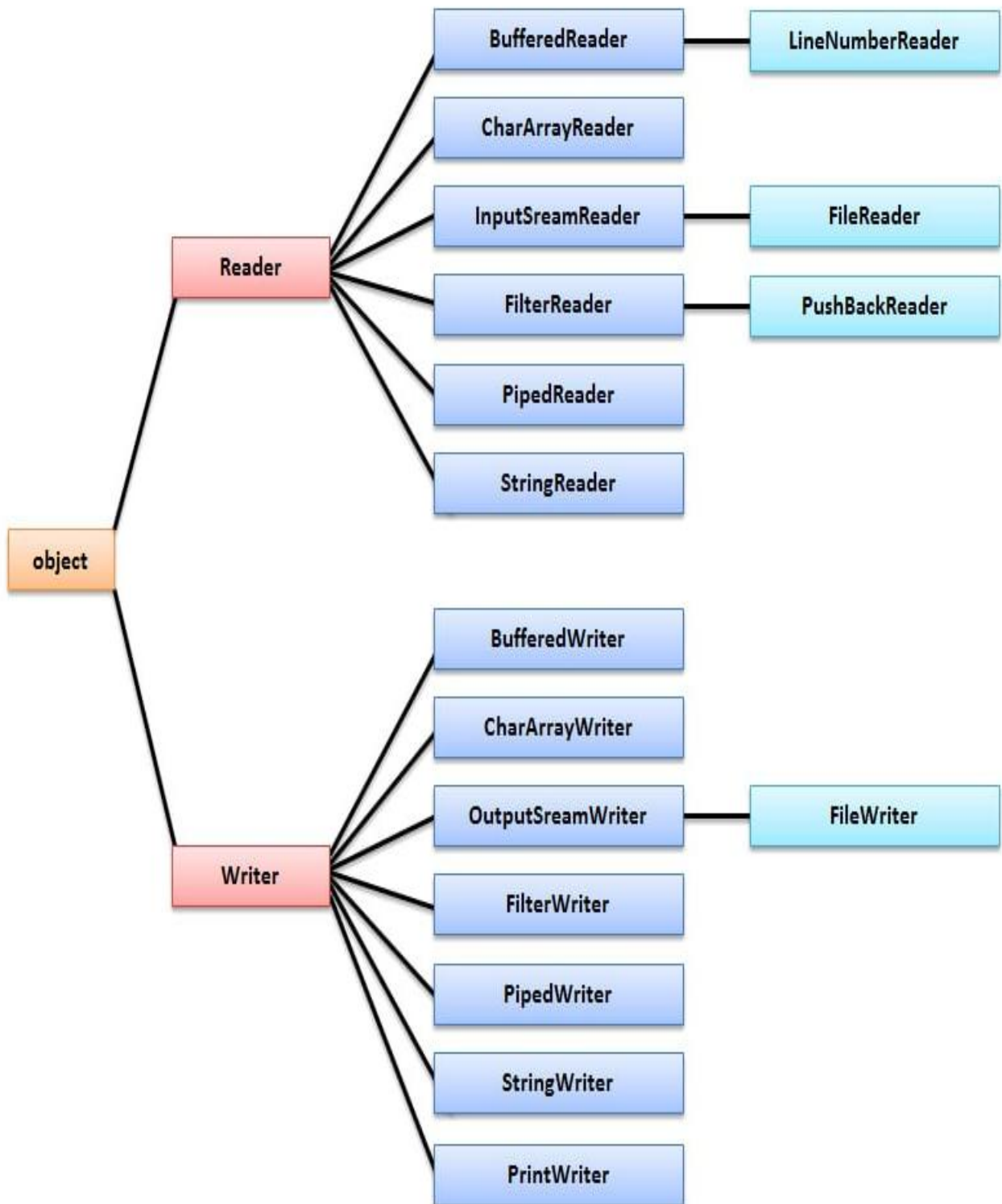
```

1  import java.io.Writer;
2  import java.io.FileWriter;
2  import java.io.BufferedWriter;
3  import java.io.IOException;
4  import java.io.Reader;
5  import java.io.BufferedReader;
6  import java.io.FileReader;
7
8
9  public class Main {
10     public static void main(String[] args)
11     {
12         //كود الـWriter
13         Writer out = null;
14         try
15         {
16             Writer inner = new FileWriter
17             ("c:\\temp\\myfile.txt");
18             out = new BufferedWriter(inner);
19
20             //write data to the file
21         } catch (IOException e)
22         {
23             e.printStackTrace();
24         }
25
26         //كود الـReader
27         Reader in = null;
28         try
29         {
30             Reader inner = new FileReader
31             ("c:\\temp\\myfile.txt");
32             in = new BufferedReader(inner);
33
34             //process file
35         } catch (IOException e)
36         {
37             e.printStackTrace();
38         }
39         finally
40         {
41             try { in.close(); }
42             catch (Exception e) {}
43         }
44     }
45 }

```



كفاحُ هذه الدنيا كفاحُ
ويهزم ليلنا دوماً صباحُ



مما سبق، نلاحظ أن جميع هذه الكلاسات تأخذ file name كargument:
 FileInputStream, FileOutputStream, FileReader, FileWriter.
 أي نمرر اسم الملف في constructor الخاص بها.

■ مثال أبسييزي لاستخدام الكلاس Reader & Writer مع ال Buffer :

```

1  // استيراد الحزم المناسبة
2  import java.io.BufferedReader;
3  import java.io.FileReader;
4  import java.io.IOException;
5
6  public class Main {
7      //
8      public static void main(String[] args)
9      throws IOException {
10         // إنشاء كائن من الـ BufferedReader
11         // لقراءة مجموعة من الملف لجعل الكود أسرع
12         BufferedReader inFile = new
13         BufferedReader (new FileReader
14         ("data.txt"));
15         // يتم تعريف متغير لتخزين السطر الحالي
16         // الذي يتم قراءته من الملف
17         String line;
18         // دالة readLine تقرأ سطرًا كاملاً من الملف
19         // النصي.
20         // تُعيد null إذا تم الوصول إلى نهاية الملف.
21         // تقرأ كل سطر من الملف وتتحقق مما إذا
22         // (null ليس) كان يحتوي على بيانات
23         // إذا وصل البرنامج إلى نهاية الملف، ستنهي
24         // الحلقة.
25         while ((line = inFile.readLine()) !=
26         null) {
27             System.out.println(line);
28         }
29         // إغلاق الملف
30         inFile.close();
31     }
32 }

```

```

1  import java.io.BufferedWriter;
2  import java.io.IOException;
3  import java.io.Reader;
4  import java.io.BufferedReader;
5  import java.io.FileReader;
6
7
8  public class Main {
9      public static void main(String[] args)
10     {
11         // كود الـ Writer
12         Writer out = null;
13         try
14         {
15             Writer inner = new FileWriter
16             ("c:\\temp\\myfile.txt");
17             out = new BufferedWriter(inner);
18
19             // write data to the file
20         } catch (IOException e)
21         {
22             e.printStackTrace();
23         }
24         finally
25         {
26             try { out.close(); } catch (Exception e) {}
27         }
28     }
29 }

```

PrintWriter :

هو كلاس في جافا ينتمي إلى الحزمة java.io ، ويستخدم لتنسيق وإرسال النصوص إلى الملفات أو المخرجات النصية (Stream للكتابة) مثل (System.out)، ويتيح الكتابة بسهولة وطريقة منسقة، حيث يتعامل مع نوعي الملفات (text/binary).

4. مميزات PrintWriter :

1. التنسيق السهل .
2. التعامل مع Buffering ويحتوي على العديد من الميثودات المشابهة لميثودات (System.out) مثل :

1. println : يكتب النص ويضيف سطر جديد .
2. print : يكتب النص دون إضافة سطر جديد .
3. flush : ضمان أن كل البيانات قد تمت كتابتها في الملف (write buffered output to disk).
4. close : إغلاق الـ PrintWriter والملف بعد الانتهاء من الكتابة.
5. format : مشابهة لـ printf تستخدم لتنسيق النصوص (للكتابة بتنسيق معين) .

مثال :

```

1. import java.io.*;
2.
3.
4. public class Main {
5.
6.     public static void main(String[] args) throws IOException {
7.         PrintWriter outFile = new PrintWriter(new FileOutputStream ("out.txt"));
8.         //إنشاء كائن من printWriter من أجل الكتابة على الملف
9.
10.        outFile.println("Hello, World!"); // كتابة نص مع سطر جديد
11.        outFile.print("This is ");      // كتابة نص دون سطر جديد
12.        outFile.println("a test.");      // استكمال النص مع سطر جديد
13.        outFile.format("Value: %.2f%n", 123.456); // تنسيق النصوص
14.
15.        outFile.flush(); // التأكد من كتابة جميع البيانات
16.        outFile.close(); // إغلاق الملف
17.    }
18. }
```

Combination of two classes:

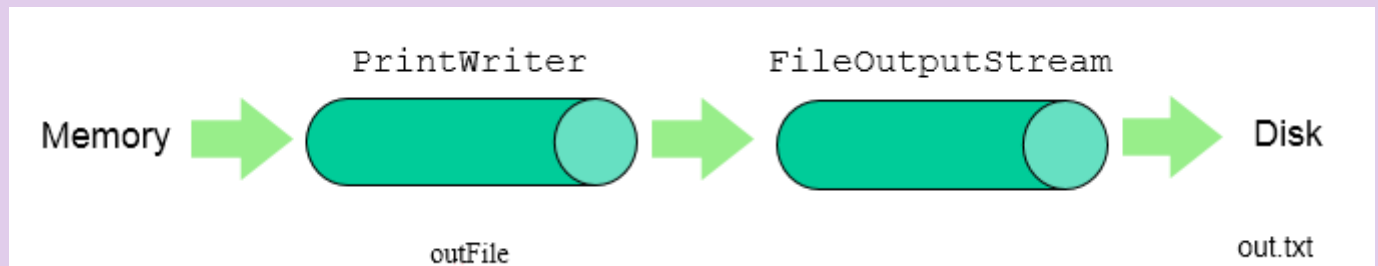
نحتاج في كثير من الأحيان إلى استخدام أكثر من كلاس، فنلاحظ أننا في المثال السابق استخدمنا `PrintWriter` مع `FileOutputStream` لكتابة البيانات إلى ملف في السطر السابع، وهذه الطريقة اختصار للطريقة الطويلة:

```
FileOutputStream s = new FileOutputStream("out.txt");
```

```
PrintWriter outputStream = new PrintWriter(s);
```

والهدف هو إنشاء object من `PrintWriter` يستخدم `FileOutputStream` لفتح text file.

أي : `FileOutputStream` "connects" `PrintWriter` to a text file.



Bridging the Gap:

نحتاج أحياناً إلى الربط بين نوعين مختلفين من الكلاسات من حيث الهيكلية (Character/Binary)، ونقوم بذلك باستخدام:

- InputStreamReader

يقوم بقراءة بايتات من `InputStream` وتحويلها إلى محارف باستخدام ترميز نصي معين (character encoding).

- OutputStreamWriter

يقوم بتحويل المحارف المرسلّة لـ `Writer` إلى بايتات باستخدام ترميز نصي معين (character encoding) وكتابتها إلى `OutputStream`.

التعامل مع `File` class في جافا

تم إعداد الكلاس `File` لمعالجة الملفات (files) و المجلدات (directories)، حيث لا يوجد كلاس خاص لمعالجة المجلدات لأنها عبارة عن ملفات فارغة، وكل ملف في نظام الملفات ضمن operating system يمكن تمثيله عن طريق هذا الكلاس الذي يحتوي على العديد من methods التي سنتحدث عنها.

- `File` class acts like a wrapper class for file names.
- A file name like "numbers.txt" has only String properties.

للاستدعاء الكلاس `File` :

```
import java.io.File;
```

■ فوائد الكلاس File :

1. إنشاء المجلدات.
2. الوصول والتلاعب بالملفات والمجلدات وقراءة محتواها .
3. التحقق من وجود الملفات .
4. إعادة تسمية أو حذف الملفات والمجلدات.
5. التحقق من خصائص وأذونات الملفات و الاستعلام عنها .

■ ملاحظة :

في حال أردنا القراءة من ملف يجب أن يكون الملف موجوداً بالأصل، لأن الـ File class وظيفته أن يؤشر على المرجع الخاص بالملف، أما في حال أردنا الكتابة على ملف ولم يكن متواجداً فمن الممكن أن ينشئ ملفاً جديداً و يبدأ بالكتابة عليه، وفي كل مرة نقوم بالكتابة عليه يقوم بـ overwrite (إزالة المحتوى القديم وكتابة الجديد).

Some Methods of File class :

3. String getName() : إرجاع اسم الملف الذي يشير إليه مسار كائن الـ File.
4. String getPath() : إرجاع مسار الملف الذي يشير إليه الكائن.
5. String getParent() : إرجاع مسار الأب للملف الذي يشير إليه كائن الـ file، أو ترجع null إذا لم يكن موجوداً.
6. boolean canRead() : تجد إذا كان البرنامج قادراً على القراءة من الملف الذي يشير إليه الكائن (الملف قابل للقراءة).
7. boolean canWrite() : تجد إذا كان البرنامج قادراً على الكتابة في الملف الذي يشير إليه الكائن (الملف قابل للكتابة).
8. boolean exists() : تجد إذا كان الملف موجوداً أم لا .
9. boolean delete() : تمسح الملف أو المجلد الذي يشير إليه الكائن، وتعيد true إن تمت العملية .
10. long length() : ترجع طول الملف (عدد المحارف الموجودة فيه) وترجع 0 إذا لم يكن الملف موجوداً.
11. boolean isDirectory() : تتحقق من كون كائن الـ File يمثل ملفاً أو مجلداً.

■ ملاحظة: يمكنك الاطلاع على javadocs للحصول على معلومات مفصلة حول جميع الـ methods فيه وكيفية استخدامها.

Scanner class :

يستخدم لقراءة البيانات من مصادر مختلفة (لوحة المفاتيح / الملفات وغيرها)

للاستفادة من هذا الكلاس يجب استيراده من الحزمة

```
import java.util.Scanner;
```

(1) Scanner scan = new Scanner (...); إنشاء كائن.

(2) قراءة البيانات

(3) Scanner.close(); إغلاق الكائن.

ويمكن أن نميز حالتين عند إنشاء الكائن:

← قراءة البيانات من لوحة المفاتيح:

```
Scanner keyboard = new Scanner(System.in);
```

← قراءة البيانات من ملف : إنشاء كائن مخصص للقراءة من ملف:

```
Scanner inFile=new Scanner (new File("in.text"));
```

ملاحظة أريسيزية: توجد خيارات أخرى للقراءة منها، كالقراءة من String أو BufferedInputStream.

مثال 1 :

```
1 import java.util.Scanner;
2 import java.io.File;
3                                     import
java.io.FileNotFoundException;
4
5 public class Main {
6     public static void main(String[] args)
7     {
8         try {
9             //يتم فتح ملف النص in.txt
10             Scanner inFile = new Scanner
(new File("in.txt"));
11             //متغير لتخزين الأرقام بعد قراءتها من الملف
12             int number;
13             // يتحقق مما إذا كان هناك رقم صحيح
14             // (integer) متاح للقراءة في الملف.
```

```
13 // إذا كان هناك رقم، تُعيد القيمة true
14 وتستمر الحلقة. إذا لم يكن هناك، تُعيد false
15 وتنتهي الحلقة.
16 while (inFile.hasNextInt()) {
17     // يقرأ الرقم الصحيح التالي من الملف
18     // ويخزنه في المتغير number
19     number = inFile.nextInt();
20     System.out.println("تم قراءة الرقم " +
number);
21 }
22 //إغلاق المصدر
23 inFile.close();
24 } catch (FileNotFoundException e) {
25     System.out.println("خطأ: الملف غير
26     موجود.");
27 }
```

مثال 2 : (لدينا ملف يمثل حساب عميل في بنك ويحتوي اسماً ورقماً صحيحاً ورقماً عشرياً)

```

1  import java.util.Scanner;
2  import java.io.File;
3  import java.io.FileNotFoundException;
4
5  //تعريف الفئة Account
6  class Account {
7      private String name;
8      private int id;
9      private float balance;
10
11     // Constructor لإنشاء كائن من نوع Account
12     public Account(String name, int id, float balance) {
13         this.name = name;
14         this.id = id;
15         this.balance = balance;
16     }
17
18     // عرض معلومات الحساب
19     public void display() {
20         System.out.println("اسم الحساب: "
21 + name + ", رقم التعريف: " + id + ", الرصيد: " +
22 balance);
23     }
24     public class Main {
25         public static void main(String[] args)
26         {
27             // القيام بـ IOException handling

```

```

27         try {
28             // فتح الملف والقراءة منه
29
30             Scanner inFile = new Scanner
31             (new File("in.txt"));
32
33             // قراءة البيانات من الملف
34             while (inFile.hasNext()) {
35                 String name = inFile.next(); // قراءة
36                 الاسم
37                 int id = inFile.nextInt(); // قراءة رقم
38                 التعريف
39                 float balance = inFile.nextFloat(); //
40                 قراءة الرصيد
41                 // إنشاء كائن جديد من نوع Account
42                 باستخدام البيانات
43                 Account account = new Account
44                 (name, id, balance);
45                 account.display(); // عرض معلومات
46                 الحساب
47             }
48
49             // إغلاق الملف
50             inFile.close();
51         } catch (FileNotFoundException e)
52         {
53             System.out.println(e);
54         }
55     }
56 }

```

الفرق بين الـ Scanner و BufferedReader

BufferedReader	Scanner
تستخدم لقراءة النصوص من المصدر بطريقة سريعة.	تستخدم لقراءة البيانات من ملف أو لوحة المفاتيح أو غيرها.
فيها : (1) <code>readLine()</code> : تقرأ سطراً كاملاً من النص حتى تصل إلى نهاية السطر أو نهاية الملف (تعيده ك string) وإذا وصلت إلى نهاية الملف تعيد null . <pre> BufferedReader inFile = new BufferedReader (...); while ((line = inFile.readLine()) != null) { // ... } </pre> (2) <code>read()</code> : تقرأ حرفاً واحداً (قيمة int) ، وإذا وصلت إلى نهاية الملف تعيد (-1) .	فيها : (1) <code>nextLine()</code> : تقرأ سطراً كاملاً من النص وعندما تصل إلى نهاية السطر تعيد النص (String)، وإذا وصلت لنهاية الملف ترمي <code>NoSuchElementException</code> (2) <code>hasNextLine()</code> : تتحقق فيما إذا كان هناك سطر إضافي متاح للقراءة . (3) <code>nextInt()</code> : تقرأ العدد الصحيح من النص، وإذا لم يكن النص يحتوي على عدد صحيح ترمي <code>InputMismatchException</code> (4) <code>hasNextInt()</code> : تتحقق إذا كان النص يحتوي على عدد صحيح متاح للقراءة. (5) <code>nextFloat()</code> : تقرأ العدد العشري من النص.
نختبر فيها نهاية الملف (EOF) بما تعيده الـ <code>method()</code> التي نستخدمها (null أو -1).	نختبر فيها نهاية الملف (EOF) بـ <code>hasNext...()</code> (نملأ الفراغ حسب نوع البيانات) ونتجنب عن طريقها رمي الاستثناءات، حيث تعيد null إن لم يعد هناك حرف/سطر/رقم قابل للقراءة.

ملخص:

- Important classes for text file **output** (to the file)
 - `PrintWriter` (يُفضل استخدامه)
 - `FileOutputStream` [or `FileWriter`]
- Important classes for text file **input** (from the file):
 - `BufferedReader`
 - `FileInputStream` [or `FileReader`]
 - `Scanner` (يُفضل استخدامه)

إضافة أريسيزية:

- لكي نحفظ object ضمن file أو نتلقى object من file يوجد لدينا العديد من الخيارات:
 - بعضها يعتمد على (chars) في قراءته أو كتابته للبيانات مثل: `fileReader` and `fileWriter` - `bufferedReader` and `bufferedWriter`
 - بعضها يعتمد على الـ (bytes) في قراءته أو كتابته للبيانات مثل: `fileInputStream` and `fileOutputStream` - `objectInputStream` and `objectOutputStream`

سنعرض إحدى الطرق التي هي char based والتي سنخزن من خلالها بياناتنا ضمن text files:

ما سنستخدمه حتى يتم وضع الأوبجكت بـ Text File هو ببساطة استخدام String يحوي كافة بيانات هذا الـ object، لكن هذا سيشكل مشكلة كوننا سنضعه كـ String و الذي هو بدوره text.

كيف سنفرق بين attributes هذا الـ object عند قرائتها مجدداً من الـ text file؟

- إما نعطي كل attribute خاصة بالـ object سطرًا ونكتبها بهذا الشكل إلى الـ file.
 - أو ببساطة سنستخدم ما نسميه الـ delimiter بحيث نضع نوعاً من أنواع الأحرف الفاصلة بين كل attribute خاصة بالـ object و بين الأخريات بنفس السطر.
- و نحن سنعرض الطريقة الثانية (لا فرق كبيراً بينها وبين الأولى لكن قد يحدث فرق طفيف جداً بالأداء - لصالح الثانية - نظراً لتقليل العمليات التي يقوم بها الـ reader أو الـ writer).

الخطوات هي التالية:

عند الكتابة:

1. ننشئ String مكون من attributes الأوبجكت ونفصل بينها بنوع ثابت من الأحرف الفاصلة.
2. نكتب هذا الـ String إلى الـ file فيصبح سطرًا.

عند القراءة:

1. نقرأ السطر من الملف فيكون String.
2. سنستخدم دالة split على هذا الـ String التي ترجع بدورها array of strings، سنمرر لها الأحرف الفاصلة التي اخترناها، ومعنى ذلك أن الـ array of strings الناتجة سيتم تقسيمها بحيث كل عنصر فيها هو أحد بيانات الأوبجكت (ستقطع الدالة الـ String عند كل تعبير فصل يظهر أمامها).
3. نسند بيانات هذه الـ Strings إلى كل attribute من attributes الأوبجكت على حدة، ونقوم بعمليات الـ parsing للبيانات لأنواعها الفعلية.
4. ننشئ أوبجكت جديد من هذه البيانات.

لو كان لدينا user that has username password and email

- ننشئ String من الشكل username—password—email فرضاً نكتب هذا السطر إلى file.
- ثم عند القراءة نقرأ السطر و نسنده لمتغير ما فيصبح:

```
string x = username—password—email
```

- نستدعي التابع لعمل array of string بناءً على attributes الاوبجكت
- String[] y = x.split ("—")
- سنحصل على مصفوفة فيها: username ، password ، email (كل منهم بـ String مستقل).
- الآن نسنده القيم لأوبجكت جديد مثلاً:

```
User user = new User(y[0],y[1],y[2])
```

- ولو كان لدينا أحد هذه البيانات يفترض أن تكون int او double مثلاً نقوم بعمل cast لها ثم نقوم بعمل الاوبجكت.

سنعرض ذلك بالكود الآن:

```
1. import java.io.*;
2. class User {
3.     private String username;
4.     private String password;
5.     private String email;
6.     public User(String username, String
password, String email) {
7.         this.username = username;
8.         this.password = password;
9.         this.email = email;
10.    }
11.    public String getUsername() {
12.        return username;
13.    }
14.    public String getPassword() {
15.        return password;
16.    }
17.    public String getEmail() {
18.        return email;
19.    }
```

```
20.    // هذا التابع سيقوم بتحويل ال object إلى
string
21.    public String toDelimitedString() {
22.        // نحول ال User إلى نص مع تحديد فاصل
من نوع ما و ليكن ---
23.        return username + "---" + password
+ "---" + email;
24.    }
25.    // هذا التابع سيقوم بتحويل صيغة ال
string إلى
object
26.    public static User
fromDelimitedString (String data) {
27.        // نقطع النص بناءً على الفاصل --- الذي اخترناه
28.        String[] parts = data.split("---");
29.        if (parts.length != 3) {
30.            throw new IllegalArgumentException
("Invalid data format.");
31.        }
32.        return new User(parts[0], parts[1],
parts[2]);
33.    }
34. }
```

```

1. public class Main {
2.     // مسار الملف
3.     private static final String FilePath =
"users.txt";
4. // هذا التابع سيقوم بالكتابة على الفايل نمر له user
5.     public static void saveUser(User user)
throws IOException {
6.         // ننشئ ال buffered writer و هو ال stream
الذي سنستخدمه للكتابة على ال file
7.         try (BufferedWriter writer = new
BufferedWriter(new FileWriter(FilePath))) {
8.             // نستدعي تابع الكتابة لكن على ال string
الناتج عن التابع الذي كتبناه مسبقاً
9.             writer.write(user.toDelimitedString());
10.        }
11.    }
12.    // تابع قراءة من الفايل
13.    public static void loadUsers() throws
IOException {
14.        // ننشئ ال buffered reader وهو ال stream
الذي سنستخدمه للقراءة من ال file
15.        try (BufferedReader reader = new
BufferedReader(new FileReader(FilePath))) {
16.            // سنخزن به سطور ال file حتى تنتهي بالتدريج
17.            String line;
18.            // طالما لم نصل لآخر سطر والذي ستكون قيمته null
19.            while((line = reader.readLine())!=null) {
20.                // قم بعمل مستخدم بناء على البيانات الموجودة
ضمن السطر الحالي
21.                User user = User.fromDelimitedString
(line);
22.                // اطبع بيانات المستخدم الذي تمت قراءتها
23.                System.out.println ("Loaded User: "
+user.getUsername()+" "+ ser.getEmail());
24.            }
25.        } }

```

```

26. public static void main(String[] args) {
27.     try {
28.         // ننشئ user
29.         User user = new User ("doaa",
"doaa123", "doaa@gmail.com");
30.         // نحفظه
31.         saveUser(user);
32.         // نقرأه
33.         loadUsers();
34.     } catch (IOException e) {
35.         System.err.println();
36.     }
37. }
38. }

```



The End